

Introducción a la ingeniería de software

¿Qué es el software?

Software es un set de programas y la documentación que acompaña. *No es solamente el código, como se suele asumir.* Con saber programar, no alcanza para poder hacer software. Existen 3 tipos básicos de software:

1. System software
 - Se encarga de controlar y gestionar los recursos del HW de una computadora, así como proporcionar una interfaz para que los usuarios interactúen.
2. Utilitarios
 - Su función principal es mejorar y optimizar el rendimiento del S.O. y del HW de la computadora.
3. Software de aplicación
 - Es utilizado para realizar tareas específicas en una computadora. Busca satisfacer las necesidades de los usuarios.

Software ≠ Manufactura

No existe manera de comparar software con manufactura, ya que:

- El software es menos predecible (es intangible)
- No hay producción en masa, ningún producto de software es igual a otro (cambian requerimientos o las necesidades de los usuarios)
- No todas las fallas son errores, los errores son tratados de forma distinta que en la producción.
- El software no se gasta (a lo sumo, pierde vigencia por cambios de requerimientos)
- El software no está gobernado por las leyes de la física

Todas estas diferencias están atadas a la **intangibilidad** del software como producto.

Problemas al desarrollar software

- La versión final del producto no satisface las necesidades del cliente.
- No es fácil extenderlo/adaptarlo. Agregar nuevas funcionalidades es casi imposible.
- Mala documentación (desactualizada e inconsistente).
- Mala calidad.
- Mas tiempos y costos que los presupuestados.

Software exitoso

Algunas de las razones por las que el software termina siendo exitoso son:

- Involucramos al usuario en el desarrollo
- Realizamos el planeamiento adecuado
- Los requerimientos se encuentran bien enunciados
- El personal es capacitado y competente
- Tenemos el apoyo de la gerencia
- Tenemos expectativas realistas

Estos son los factores de éxito dentro del desarrollo de software. Se busca hacer que estos quedaran embebidos en el proceso de desarrollo.

Uno de los aspectos claves son los requerimientos, no solo con dejarlos claros, si no también incentivar a que el usuario tenga participación en la definición de estos.

Es muy importante la retroalimentación, para ir midiendo el avance del sistema y contar con información para poder corregir fallas o errores en el sistema.

Software no exitoso

Resulta en costos más elevados y surgen, casi siempre por no tener en cuenta los factores de éxito del desarrollo de software. Es por esto por lo que se dice que no es suficiente saber programar para hacer software, sino que hay muchas cosas a considerar.

Ingeniería de software

“Construcción de software de múltiples versiones por múltiples personas”

Es la disciplina que aplica modelos de la ingeniería y gestión para producir software:

- De calidad
- Sostenible y mantenible
- Colaborativo
- Controlado

Disciplinas de la ingeniería de software

Disciplinas técnicas

- 1. Requerimientos**
- 2. Análisis y Diseño**
- 3. Construcción**
- 4. Prueba**
 - Diseñar y ejecutar tests (unitarias, integración, sistema, aceptación). En ágil se practica testing concurrente, para detectar fallas temprano.
- 5. Despliegue**
 - Preparar y poner versiones en producción (release/deploy).

Disciplinas de gestión

- 1. Planificación de Proyecto**
 - Definir cronograma, recursos, riesgos y “triple restricción” (alcance, tiempo y costo). Es la aplicación de conocimientos, herramientas y técnicas para cumplir los requerimientos del proyecto.
- 2. Monitoreo y Control de Proyectos**
 - Seguir el plan, medir avance y calidad, gestionar desvíos y riesgos.

Disciplinas de soporte

- 1. Gestión de Configuración de Software (SCM)**
 - Mantener la integridad y trazabilidad de artefactos (código, docs, pruebas). Actividades clave: identificación de ítems, control de cambios, informes de estado y auditorías (física y funcional).
- 2. Aseguramiento de Calidad**
 - Conjunto de prácticas para garantizar que procesos y productos cumplan estándares: políticas, revisiones, auditorías, mejoras. (Las auditorías funcional y física ayudan a verificar cumplimiento de requerimientos y base documental.)
- 3. Métricas**
 - Medir proceso y producto (defectos, cobertura, lead time, velocidad, performance, etc.) para tomar decisiones y mejorar continuamente.

Ciclos de vida

Un ciclo de vida es una abstracción. Define cuáles son las **etapas** y el **orden** de cada una de ellas. No dice que tengo que hacer ni quien, ni con que herramientas ni los procedimientos, sino **solo las fases y en qué orden se van a ejecutar**.

Son la serie de pasos a través de los cuales un producto o proyecto progresa.

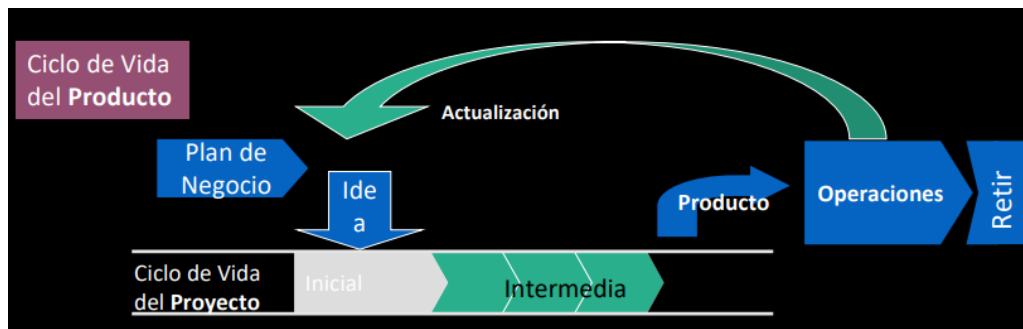
Proyecto

Es una unidad de gestión/organización del trabajo para obtener un producto. Es el medio para administrar personas y recursos y llegar a un resultado.

Al ser simplemente un medio, requiere cosas del proceso, es decir, usa el proceso, las herramientas y las personas.

Presenta un resultado único, un esfuerzo temporal definido (fecha inicio y fin) y elaboración gradual (el objetivo se descompone en otros más chicos). Este proyecto debe ser dirigido por alguien capaz.

Relación entre el ciclo de vida del proyecto y del producto



El ciclo de vida del producto es más largo y amplio, incluye todas las etapas desde que surge la idea hasta que es retirado, mientras que el ciclo de vida del proyecto es más corto y específico, abarca desde que se inicia el desarrollo hasta que se entrega el producto.

Un proyecto construye un producto, pero el producto sigue vivo más allá del proyecto, ya que puede tener nuevas versiones o mantenimiento, gestionados en proyectos distintos.

El ciclo de vida del proyecto siempre tiene inicio y fin claros, el del producto no.

Diferencia entre ciclo de vida y proceso

La principal diferencia recae en que el ciclo de vida le va a decir al proyecto, como (orden, tiempos) va a utilizar los procesos, mientras que el proceso es la definición teórica de los pasos a seguir, cosas a utilizar. Se puede decir que el proceso es una implementación del ciclo de vida, realizada para construir un producto de software.

Clasificación de los ciclos de vida

- Secuencial
 - Ejecutan una etapa después de otra sin retorno generalmente
 - Por ejemplo, el ciclo de vida en cascada.
- Iterativo/incremental
 - Se hace en iteraciones con el objetivo de ganar información y convertirla en un incremento que será incorporado en el sistema.
 - Implementados en su mayoría por procesos empíricos ya que dan la característica de adaptación y mejora del proceso. En cada iteración se capitaliza lo aprendido y se mejora la experiencia del equipo.
 - Por ejemplo, el ciclo iterativo incremental del PUD
 - A su vez, pueden ser de alcance fijo o de duración fija.
- Recursivo
 - Se utilizan por lo general en proyectos de alto riesgo.
 - Se basa en tomar una característica específica y en una iteración centrarse en esa funcionalidad, en la siguiente en otra y así.
 - Por ejemplo, el ciclo de vida en espiral

Elección de un ciclo de vida (ventajas y desventajas)

La elección del ciclo de vida depende de cuán claros y estables estén los objetivos, del nivel de incertidumbre y cambios esperados, del grado de regulación y de la madurez del equipo y del cliente.

En líneas generales, los enfoques secuenciales ofrecen previsibilidad pero son más rígidos ante cambios. Los iterativos/incrementales dan aprendizaje y valor temprano con entregas frecuentes y ajuste continuo del alcance, a costa de menor previsibilidad inicial. Los recursivos priorizan reducir incertidumbre crítica antes de invertir fuerte, pero requieren más disciplina y experiencia.

Esa elección impacta directamente en la administración: con un ciclo secuencial se planifica casi todo desde el comienzo, el presupuesto y el cronograma tienden a ser más fijos y los cambios pasan por aprobaciones formales, con uno iterativo, se planifica por etapas cortas, el presupuesto y el alcance se ajustan por tramos, hay participación frecuente de usuarios y el seguimiento se centra en valor entregado por iteración.

En síntesis: si el problema es estable y regulado, conviene mayor secuencialidad, si hay incertidumbre o necesidad de validar rápido con usuarios, conviene más iteración, si el riesgo técnico o de negocio es alto, conviene un enfoque recursivo que lo ataque primero.

Procesos de Desarrollo Empíricos vs. Definidos

Un proceso es un conjunto **estructurado** de actividades que se llevan a cabo para desarrollar un sistema de software.

Es la definición teórica de pasos a seguir y recursos a usar para desarrollar un sistema de software. Las actividades dependen de la organización y el tipo de sistema que debe desarrollarse.

- **Sw-CMM:** Es la definición teórica de un conjunto de actividades, métodos, prácticas y transformaciones que se deben seguir para desarrollar o mantener software y sus productos asociados.

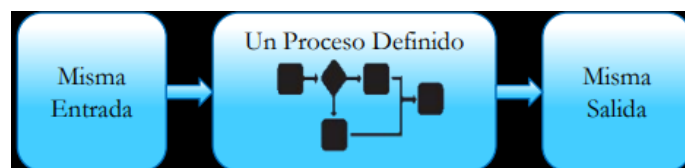


- **Procedimientos y métodos:** definición de cómo vamos a llevar a cabo las actividades de desarrollo
- **Personas:** con habilidades, entrenamiento y motivación
- **Herramientas y equipos**

Las personas hacen uso de las herramientas, equipos, procedimientos y métodos y a partir de estos producen un producto de software. Dentro de este proceso están definidas las responsabilidades, actividades y herramientas a partir de las cuales las personas van a transformar los requerimientos en software.

Procesos definidos

Asume que se puede repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados. La administración y el control provienen de la predictibilidad del proceso definido. Debe haber alguien a cargo de mantener la definición.



Un ejemplo de proceso definido es el proceso unificado de desarrollo. Estos procesos pueden utilizar todos los ciclos de vida.

Procesos empíricos (centrados en capitalizar la experiencia de los actores)

Asume que las variables son cambiantes, por lo que cuando se repite el proceso, se pueden obtener resultados diferentes. **La experiencia no es llevada de un proyecto a otro**, esta es propia de cada equipo. No se hace referencia a la experiencia de conocimiento.

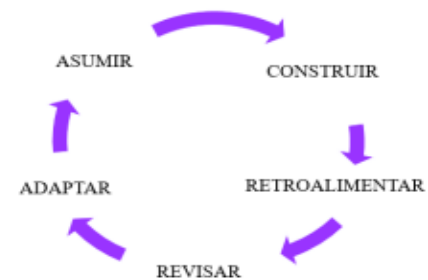
En estos procesos, el equipo tiene la potestad de resolver las cosas por sí solo, se necesita gente capaz de autogestionarse, son complicados si no se tiene experiencia.

La administración y el control se realiza a través de inspecciones frecuentes y adaptaciones (ciclos de retroalimentación), donde se toma información, se contrasta con una hipótesis y se adapta el proceso para mejorarlo.

Se centran en las personas, las claves del empirismo. Son procesos que trabajan bien y con procesos creativos y complejos.

Patrón de conocimiento de procesos empíricos

Se comienza con una hipótesis (**asumir**), se comienza a **construir** sobre esa hipótesis y posteriormente, en base a algunos puntos contruidos, podemos analizar, tomar mediciones y aprender de lo hecho (**retroalimentar**). Luego se revisa el resultado de lo construido, se contrasta con la hipótesis inicial y se realizan los cambios necesarios (**adaptar**).



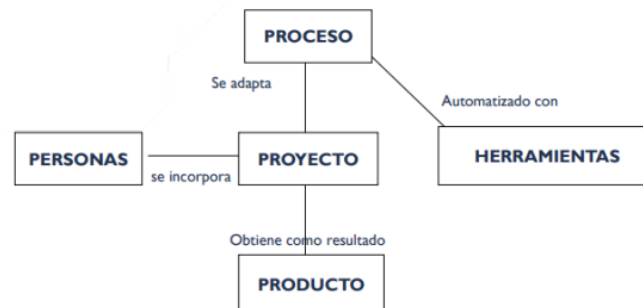
Esto se basa en una necesidad de repetición, por lo que solo es posible usar ciclos de vida iterativos e incrementales. Algunos ejemplos de procesos definidos son LEAN y AGILE.

Pilares del empirismo

1. Transparencia
 - Permite que funcione el empirismo, debemos ser sinceros, preguntar si algo no lo sabemos, compartir información con compañeros de equipo y conocimientos. Se refiere a la comunicación abierta y honesta.
2. Inspección
 - Debemos ver lo que hacemos de forma regular y sistemática para aprender de eso y detectar desviaciones y problemas.
3. Adaptación
 - Es necesario ser capaz de adaptarse y ajustar el trabajo y el proceso para realizar los cambios necesarios para corregir desviaciones y problemas, que surgen a partir de la inspección.

Componentes de proyecto de desarrollo de software

Se suele decir que un proyecto es una instanciación del proceso definida en un ciclo de vida.



En la imagen, se ve que un proyecto es llevado a cabo por personas, que implementan herramientas para automatizar los procesos y que todo esto obtiene como resultado un producto. El proceso debe adaptarse al proyecto según las necesidades (tiempo, recursos, complejidad y riesgos)

El proceso es una plantilla, materializada a través de los proyectos. En el proceso se expresa de forma teórica lo que se debe hacer para hacer software. Estos procesos toman como entrada requerimientos y a través de un conjunto de actividades obtienen como salida un producto o servicio de software.

El proyecto es la unidad de gestión/organización del trabajo para obtener un producto. Es el medio para administrar personas y recursos y llegar a un resultado. Al ser un medio, requiere cosas del proceso (lo usa), de las personas, herramientas. Este proyecto debe planificarse, tiene un objetivo (alcanzable). Define los roles de los integrantes del equipo y se debe poder medir.

Características de un proyecto

- Duración limitada
- Orientación a un objetivo
 - Los proyectos deben estar dirigidos a obtener resultados y el cumplimiento de estos se refleja a través de objetivos. Estos guían al proyecto, no deben ser ambiguos y deben ser claros y alcanzables.
- Tareas interrelacionadas basadas en esfuerzos y recursos
 - Las tareas dependen una de otra, tienen precedencia y a su vez tienen designados recursos y esfuerzos, convirtiendo la gestión en una tarea compleja.
- Son únicos

Gestión tradicional de un proyecto

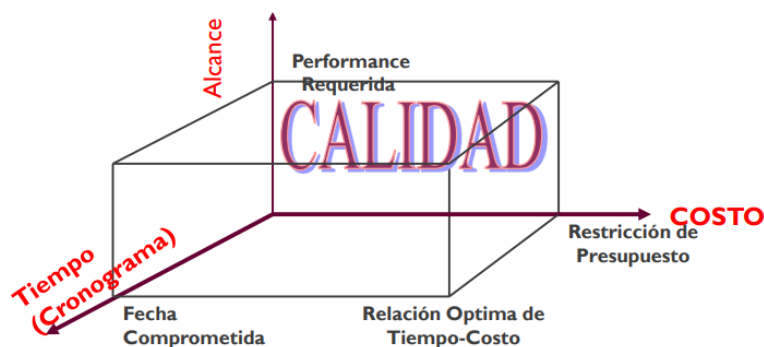
Incluye la definición del alcance/requerimientos (del producto), establecer objetivos claros y alcanzables (para el proyecto) y adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados.

El alcance de un producto se mide en términos de requerimientos de software, el de un proyecto en términos de tareas.

Triple restricción

Es una de las bases de la gestión tradicional de un proyecto. Nos dice que en un proyecto, vamos a tener una restricción de tiempo (cronograma) con fecha comprometida, un costo (presupuesto) para alcanzar los objetivos y un alcance que es lo que se va a construir.

Por detrás, de una forma no negociable, se encuentra la calidad del producto que se va a construir.



Las tres variables, están íntimamente relacionadas, de forma que si quiero reducir el tiempo, necesariamente voy a tener que hacer variar alguna de las otras variables o ambas para mantener la calidad del producto.

Es responsabilidad del PM balancear los tres objetivos involucrados dentro del proyecto.

A pesar de que se puede modificar una de las variables, produciendo un cambio en las otras, esto no quiere decir que por ejemplo, con un presupuesto ilimitado, vamos a poder hacer algo imposible. El ejemplo claro es que 9 mujeres no van a poder tener 1 hijo en 1 mes.

Roles dentro de la gestión tradicional de proyecto



En la gestión tradicional, nuestro equipo está formado por un equipo propiamente dicho y un líder de proyecto (PM). Todos son técnicos, y los clientes y jefes se encuentran por fuera.

Líder de proyecto – Project Manager

Es quien se ocupa de todas las tareas de gestión que hacen que el equipo de proyecto sea guiado a lograr el objetivo. Es el encargado de administrar todos los recursos, organizar el trabajo de las personas involucradas y hacer el seguimiento del cumplimiento de lo planeado.

Administrar un proyecto incluye poder definir los requerimientos, el alcance del producto, establecer objetivos claros y alcanzables y adaptar las especificaciones. Desde el principio, debe saber que tenemos que construir y que es lo que más pesa o lo más importante dentro del proyecto, si cumplir con el tiempo, no superar el costo o no tocar el alcance.

A su vez, es quien asigna las tareas y quien controla la triple restricción. Estas son sus responsabilidades, pero no significa que obligatoriamente las tiene que hacer el.

Equipo de proyecto

Se trata de un grupo de personas comprometidas en alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables. Todo el equipo apunta a perseguir el mismo objetivo, **alcanzable y medible**.

Para poder llegar a cumplir este objetivo, el equipo de proyecto debe **complementarse** para hacer la diferencia entre un trabajo en equipo y uno individual. A su vez, **deben poder trabajar en equipo** sin problema, por lo que por lo general se usa un grupo pequeño.

Plan de proyecto

Es el artefacto más importante de la planificación del proyecto. Debe funcionar como una **hoja de ruta**. Cuando nos sentamos frente a un proyecto, se debe delinear este plan, que no va a estar completamente definido pero va a esbozar lo general del proyecto.

Documenta lo que se hace, cuando se hace, como se hace y quien lo va a hacer.

La idea de este plan es que sea algo vivo a lo largo del proyecto, que se vaya modificando y corrigiendo a medida que pasa el tiempo. Siempre y cuando se mantenga actualizado, va a ser de uso.

Planificación de proyectos de software

Implica una serie de tareas que se deben realizar:

Definición del objetivo

Abarca lo que queremos lograr con el proyecto. Tiene que cumplir dos características muy importantes, **claridad** (bien definido, entendible por todos) **y alcanzabilidad** (posible en ese tiempo con esos recursos).

Definición del alcance

1. Alcance del producto
 - Son todas las características que pueden incluirse en un producto o servicio.
 - El cumplimiento se mide contra la especificación de requerimientos.
2. Alcance de proyecto
 - Es todo el trabajo y **solo el trabajo** que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.
 - Su cumplimiento se mide contra el plan de proyecto.

Primero se debe definir el alcance del producto, para así definir el alcance del proyecto.

Definición de proceso y ciclo de vida

Al momento de inicio del proyecto, es necesario definir qué proceso se quiere usar y que ciclo de vida.

El proceso es el conjunto de actividades que se necesitan ejecutar para construir el producto y el ciclo de vida es la manera de ejecución de esas actividades.

Dentro de la gestión tradicional, se puede elegir cualquier ciclo de vida, mientras que en agile, hay ciertas restricciones respecto al ciclo de vida.

Estimación

Sirve para prever situaciones. Se va a estimar todo antes de hacerlo, ya que no se tiene certeza de nada. Las características por estimar, necesarias para la planificación son:

1. Tamaño (producto, en cantidad de CU o requerimientos) (es el que)
2. Esfuerzo (horas hombre lineales) (es el cómo)
3. Calendario (es el cuándo)
4. Costo (es el cuanto)
5. Recursos críticos (no siempre se estiman)

Gestión de riesgos

Un riesgo es un problema esperando suceder o un evento que podría comprometer el éxito del proyecto.

Lo que se debe hacer es identificar los riesgos más probables de ocurrir y los que más podrían llegar a impactar el sistema, ya que es imposible estimar todos los riesgos.

Primero se obtiene la exposición del riesgo (en base a probabilidad de ocurrencia e impacto) y en base a esto se determina cuáles son los que se van a gestionar.

Una gestión reactiva de los riesgos implica reaccionar a estos cuando ya ocurrieron, mientras que por otro lado, una gestión proactiva sigue el ciclo de identificación, análisis, plan, seguimiento y control y aprendizaje.

Asignación de recursos

Consiste en determinar qué personas, herramientas, equipos, materiales y tecnologías se necesitan para llevar adelante las actividades del proyecto.

Programación de proyectos

La programación de proyectos es la organización temporal de todas las actividades necesarias para cumplir el alcance y los objetivos definidos.

Definición de métricas

Las métricas nos van a permitir darnos cuenta si el proyecto (durante su ejecución) está en línea con lo planificado o si está desviado. Nos permiten saber si se está cumpliendo lo planificado o no.

1. Métricas de proceso
 - a. Permiten saber independientemente de un proyecto en particular, si estamos trabajando bien o mal.
 - b. Ejemplo: porcentaje de proyectos que terminan en tiempo y porcentaje que no.

2. Métricas de proyecto

- a. Permiten saber si el proyecto se está cumpliendo de acuerdo con lo planeado.
- b. Ejemplo: comparación de tiempo real con lo planeado.

3. Métricas de producto

- a. Miden características que tienen relación directa con el producto que se está construyendo
- b. Ejemplo: métricas de tamaño (casos de uso cumplidos)

Las métricas básicas son de tamaño, esfuerzo, tiempo (calendario) y defectos (errores).

Las métricas deben ser tomadas en base a decisiones sobre el tiempo que va a tardar en realizarse el proyecto. El objetivo es minimizar esfuerzos a la hora de tomar métricas, no tiene sentido medir en milímetros algo que está a kilómetros.

Para saber si tomar una métrica o no:

1. Nos da más información que la que tenemos ahora?
2. Es la información de beneficio práctico?
3. Nos dice lo que queremos saber?

Hay casos en los que se puede dejar de tomar una métrica si ya comprobamos a lo largo de varias mediciones que siempre nos da resultados positivos.

Monitoreo y control

El Project Manager será el encargado de trabajar sobre lo definido en el plan de proyecto y determinar si se está cumpliendo o no. Para esto **se basa en lo hecho, el plan de proyecto y las métricas.**

Se compara lo planificado con lo real. No existe nunca que no haya desvío entre lo planificado y lo real, puede ocurrir que se haya hecho una sobreestimación de lo que se iba a tardar o que algo se está ocultando.

“Un proyecto se atrasa de a un día a la vez”, quiere decir que si se pueden corregir los desvíos a tiempo, se puede cumplir el objetivo.

Filosofía ágil

Es una ideología que cuenta con un conjunto definido de principios que guían el desarrollo del producto. Busca encontrar un equilibrio entre procesos definidos y nada de procesos, así como una menor cantidad de documentación.

Ser ágil no es ser indisciplinado, es una ideología bastante estricta

El manifiesto ágil se sustenta en los procesos empíricos que tienen como base la experiencia, la cual sale del propio equipo. Debido a esto, es necesario que existan ciclos de retroalimentación cortos.

Este enfoque es únicamente compatible con ciclos de vida iterativos y se basa en la idea de que la incertidumbre y los cambios son inevitables en el desarrollo de software, por lo que es necesario trabajar de manera flexible y adaptativa.

Manifiesto ágil

Es un acuerdo firmado por varias personas sobre los 4 valores ágiles especializados posteriormente en 12 principios.

1. Valorar más los individuos y las interacciones que el proceso y las herramientas
 - Enfatizar vínculos, comunicación y colaboración entre miembros del equipo, que la adopción de herramientas que se van a usar y el proceso a aplicar.
2. Valorar más lo que es de valor al cliente que la documentación extensiva
 - Enfatizar el software funcional y de alta calidad, sin dejar de lado la documentación, pero sin documentar cada detalle.
 - La documentación debe ser generada solo cuando haga falta y lo que se haya definido en un momento que debía quedar documentado.
3. Valorar más la colaboración con el cliente que las relaciones contractuales
 - Enfatizar trabajar con el cliente y comprender sus necesidades en lugar de simplemente negociar contratos y acuerdos formales.
 - Es fundamental que el cliente esté dispuesto a participar, ya que si no, no se puede aplicar la filosofía ágil.
4. Valorar más la respuesta ante el cambio que seguir con un plan estructurado inicial
 - Enfatizar ser flexible y adaptativo en la respuesta a cambios y desviaciones en lugar de seguir un plan rígido y predefinido.
 - Se debe construir en conjunto con el cliente, comenzando con una idea del producto y darle la posibilidad al cliente de cambiar de opinión.

Uno de los principios más importantes es que la forma más eficiente de comunicación es el cara a cara.

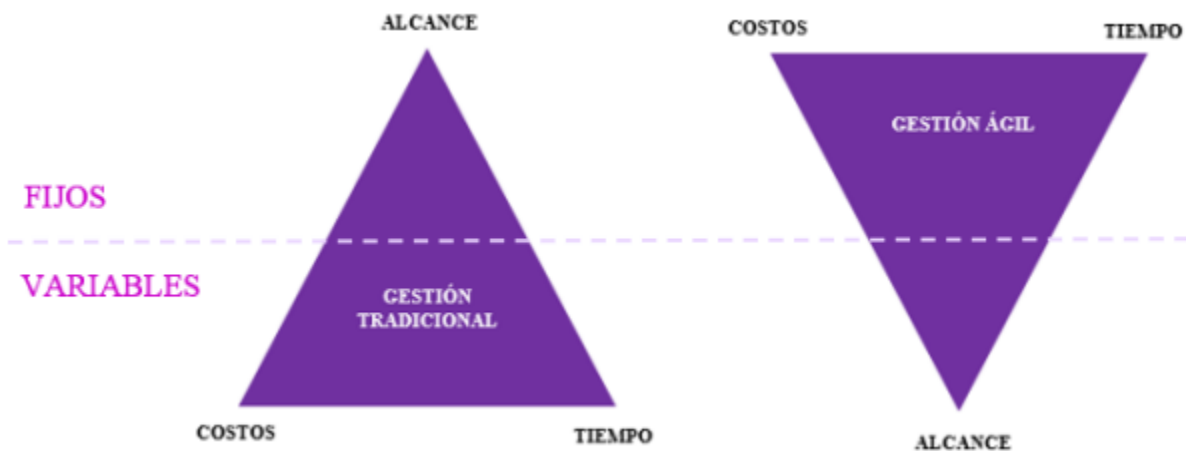
Triangulo de hierro vs Triángulo ágil

El triángulo de hierro es utilizado en la gestión tradicional. Se busca establecer un plan detallado y fijo en cuanto a los elementos necesarios antes de comenzar un proyecto y se trabaja para asegurar que se cumplan en el transcurso del proyecto variando costos y tiempo. No es positivo tener los requerimientos y alcances tan fijos (siempre todo cambia).

Por otro lado, en las metodologías ágiles, se considera a los costos y tiempo fijos (por cada sprint, entre el sprint puede variar), es decir, en este tiempo de iteración cobramos tanto dinero, pero dejando los alcances a elección del cliente. Esto busca negociar lo máximo posible el alcance con el cliente. En base a los recursos y al tiempo, te puedo definir cual es el alcance que te voy a dar.

Al cliente no le gusta esta forma de trabajo porque no puede manejar ese cambio de presupuestos cada vez que quiere un nuevo alcance, pero es positivo, ya que en la vida real, el 50% de los requerimientos son emergentes (surgen durante el desarrollo), otro 25% son conocidos identificados y el otro 25% conocidos no identificados.

El triangulo de hierro esta dirigido por el plan, mientras que el ágil, esta dirigido por el valor.



Requerimientos ágiles

En Agile, el **valor** de negocio y la construcción del producto correcto son fundamentales. Se plantea que los requerimientos se irán descubriendo a medida que avanza el proyecto y se busca definir **solo lo suficiente** para comenzar a trabajar con lo mínimo y necesario y obtener retroalimentación lo antes posible.

Básicamente, en lugar de especificar todos los requerimientos desde un inicio, el desarrollo está enfocado en trabajar en conjunto con el cliente.

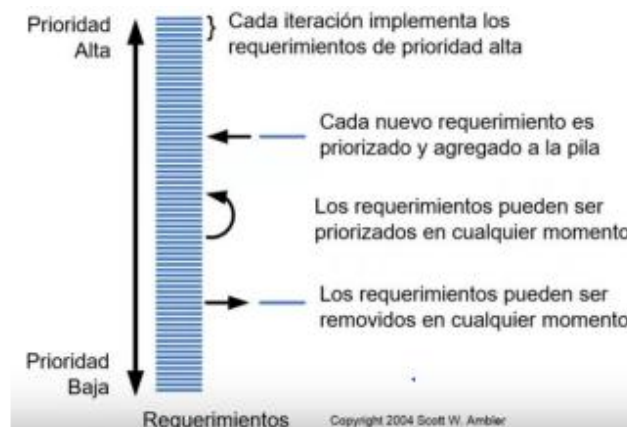
Los requerimientos son expresados en forma de historias de usuario. Estas permiten definir el alcance de cada iteración del proyecto y se priorizan en función del **valor** que aportan al producto.

Una de las razones por las cuales esto se comienza a hacer así, es que todos los productos de software tienen un desperdicio, ya que de todas las funciones desarrolladas, un pequeño porcentaje son las usadas por el cliente.



Para contrarrestar esto, se priorizan solo las que aportan valor al cliente. Para esto, se define el concepto de PO. El PO es el responsable de identificar las necesidades y prioridades del negocio y en consecuencia, priorizar los requerimientos del producto.

Todos estos requerimientos ágiles son organizados en una lista priorizada y ordenada llamada **product backlog**. Los requerimientos más importantes (los que aportan más valor) se encuentran en la parte más superior. El PO es quien puede agregar/modificar/eliminar cosas del product backlog.



Just In Time

Analizo cuando lo necesito, no antes ni después. Analizo con detalle un requerimiento cuando tenga suficiente valor.

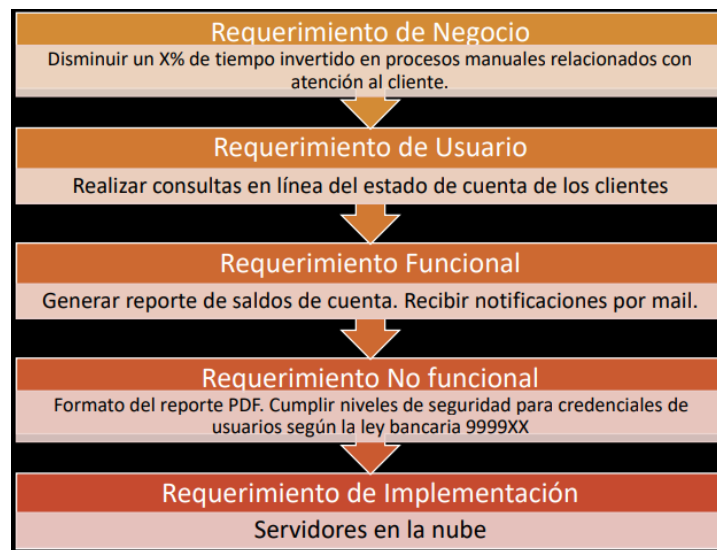
Este concepto se usa fundamentalmente para evitar desperdicios (esencialmente de tiempo). No conozco el 100% de los requerimientos en un inicio, por lo que los describo a medida que hacen falta.

Lo que debemos entregarle al cliente es valor de negocio, no características de software. El software es el medio a través del cual nosotros le damos valor al negocio.

Tradicional vs Ágil

	TRADICIONAL	ÁGIL
Prioridad	Cumplir el plan	Entregar Valor
Enfoque	Ejecución ¿Cómo?	Estrategia (¿Por qué? ¿Para qué?)
Definición	Detallados y cerrados. Descubrimientos al inicio	Esbozados y evolutivos. Descubrimiento progresivo
Participación	Sponsors, stakeholder de mayor poder e interés.	Colaborativo con stakeholders de mayor interés (clientes, usuarios finales)
Equipo	Analista de negocios, Project Manager y Áreas de Proceso.	Equipo multidisciplinario.
Herramientas	Entrevistas, observación y formularios	Principalmente prototipado. Técnicas de facilitación para descubrir.
Documentación	Alto nivel de detalle. Matriz de Rastreabilidad para los requerimientos.	Historias de Usuario. Mapeo de historias (Story Mapping)
Productos	Definidos en alcance	Identificados progresivamente
Procesos	Estables, adversos al cambio.	Incertidumbre, abierto al cambio

Tipos de requerimientos



A raíz de esta clasificación, queremos identificar aquellos requerimientos que tienen que ver con el dominio del problema y luego de la solución y por ende con los requerimientos específicos de software. Esta distinción es muy importante ya que es fundamental entender las necesidades del usuario y las restricciones del contexto en el que se utiliza el software.

En resumen, en los requerimientos ágiles se busca entender qué le da valor al negocio, cuáles son los requerimientos de negocio y a raíz de estos construir una solución que pueda entregarse al cliente, priorizando lo que le da valor al mismo y a partir de entregas continuas.

Conceptos relacionados a los principios ágiles

- Los cambios son la única constante.
- Debemos escuchar a todos los que tengan algo para decir del producto (stakeholders).
- El usuario dice lo que quiere cuando recibe lo que pidió.
- No hay técnicas ni herramientas que sirvan para todos los casos.
- Lo importante no es entregar una salida, si no una solución que aporte **valor**.

Principios ágiles relacionados a los requerimientos ágiles

1. La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes
2. Recibir cambios de requerimientos, aun en etapas finales.
4. Técnicos y no técnicos trabajando juntos todo el proyecto: El PO es parte del equipo, porque si los no técnicos no son parte del equipo del proyecto, difícilmente pueda identificar qué da valor al negocio
6. El medio de comunicación por excelencia es cara a cara.
11. Las mejores arquitecturas, diseños y requerimientos emergen de equipos auto organizados

User Stories

Son una técnica para capturar los requerimientos, como si se estuviese contando una historia. No son especificaciones detalladas de requerimientos (como los CU), si no que expresan la intención de hacer algo. Necesitan poco o nulo mantenimiento y pueden descartarse luego de la implementación.

En general, no tienen demasiado detalle al principio del proyecto y son elaboradas evitando especificaciones anticipadas.

Tres “C” de una User Story

Conversación

Es la parte más importante que no queda escrita en ningún lugar. Es la parte no visible. Es el momento en el cual uno dialoga con el PO para captar las necesidades del problema. Es donde se obtienen todos los detalles para escribir la User Story y comenzar a trabajar en ella.

Confirmación

Son las condiciones que se tienen que dar para que la US este satisfecha.

Tarjeta (card)

Es la parte visible de la US. Es donde escribimos o expresamos algo que nos indique cual es la historia de usuario, de que se trata y su valor de negocio.

Forma de expresar una US

COMO <NOMBRE DE ROL> QUIERO <ACTIVIDAD>

PARA <VALOR DE NEGOCIO QUE RECIBO>

A su vez, esta US debe ser expresada con una frase verbal, que permita identificar de que se trata o que va a tener esta.

El rol no representa a una persona, si no lo que puede estar haciendo una persona.

Las US son multipropósito porque modelan o representan distintas cosas:

1. Una necesidad de un usuario.
2. Una descripción del producto.
3. Un ítem de planificación (se prioriza en el Product Backlog)
4. Token para una conversación
5. Mecanismo para diferir una conversación

El PO es el encargado de priorizar y ubicar las US en el product backlog. Esta es la lista de todos los requerimientos que tiene que cumplir mi producto. Las US que están más arriba en la pila, son las que tienen que ser modeladas con mayor nivel de detalle y a las que se les debe prestar un mayor nivel de atención.

Las US deben estar planteadas como una implementación en porciones verticales. No se trata de escribir o detallar la funcionalidad que necesita el usuario, también se debe implementar desde la lógica de la interfaz de usuario hasta como se resuelve en la BD.

Criterios de aceptación

Son aquellas cuestiones que le definen límites a la US. Ayudan a que el PO responda los criterios que necesita para que la US le provea valor al negocio.

Estos criterios no contienen cuestiones de implementación, tienen que ver con la perspectiva del usuario. Siempre se define una intención, no se define una solución.

A su vez, estos criterios ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad y a los testers a derivar las pruebas pertinentes.

La redacción debe ser clara y tienen que haber validaciones concretas. Se usan las palabras PUEDE (opcional) y DEBE (obligatorio).

Pruebas de aceptación

Se encuentran en el dorso de la tarjeta, complementan la historia y expresan detalles de la conversación. Nos dan la confirmación de que finalmente lo que estamos haciendo, va a ser lo que el PO espera. Tiene que ser algo que el usuario **PUEDA** probar.

Agrega también lo que esperamos que suceda cuando ejecutamos la US, entre paréntesis se agrega si PASA o FALLA, refiriéndonos al resultado de la prueba.

Definición de listo

Permite definir que tiene que cumplir una US para asegurarnos de que esta puede ser implementada. Es definida por el equipo.

Normalmente, es una definición que acuerda el equipo y en base a esto se arma una checklist a partir de la cual se toma una US y se determina si se cumple con los aspectos acordados. Si se cumple con todo, puede ser incluida en un sprint, pero si no, es necesario trabajar más en esa US para poder incluirla en el próximo sprint.

Que nos ayuda a definir cuando una US esta lista?

Si la US cumple con el modelo INVEST, por lo general se considera lista. Es una manera de trabajar sobre la Definition of Ready

I: INDEPENDIENTE	Que no dependa de otra US, poder incluirla en una Sprint porque total no depende de otra US, puede ser implementable en cualquier orden. No me afecta a otra US mover mi US dentro del Product Backlog
N: NEGOCIABLE	Se negocia el QUÉ, no el CÓMO; por esto es que decimos que en los criterios de aceptación no ponemos detalles de implementación, lo que negociamos con nuestro cliente es el QUÉ. Si la user define el COMO está mal definida
V: VALUABLE	Debe tener un valor concreto para el cliente
E: ESTIMABLE	Tengo que poder definir cuánto esfuerzo me va a llevar hacer la US, para ayudar al cliente a armar un ranking basado en costos.
S: SMALL	La US tiene que entrar en una Sprint. Para saber si es small, tengo que poder estimarla.
T: TESTEABLE	Tengo que poder demostrar que fueron implementadas.

Definición de hecho

Es otra checklist propia del equipo, donde se destacan las características que debe tener la US y lo que indica si esta decentemente terminada para presentársela al PO o cliente. Debe tener las pruebas unitarias y de aceptación en verde, tiene que haber pasado el ambiente de prueba, etc.

Niveles de abstracción

Las épicas son historias de usuario muy grandes, que en principio son así de grandes porque están en un lugar de la pila donde todavía no fueron detalladas. Como todavía no deben ser implementadas, se dejan plasmadas en una gran US, sabiendo que en algún momento voy a tener que llegar a disolverla, detallarla y hacerla más pequeña.

Los Temas son más grandes que una épica. Son un conjunto de US relacionadas que se definen en función de un título para recordarme que todo lo que está incluido en este tema deberá ser tratado en algún momento. Tienen un nivel de abstracción muy grande.

Los SPIKES son un **tipo especial de US**, que sirven para quitar riesgo o incertidumbre de otro requerimiento, de otra US o de alguna faceta del proyecto que nosotros queremos investigar. Son específicamente creadas para esto. Por ejemplo, investigar una tecnología que no conocemos. Pueden ser técnicas (investigar enfoques técnicos del dominio de la solución) o funcionales (cuando hay incertidumbre respecto de cómo el usuario interactuara con el sistema). Siguen siendo una entrega de valor, pero de conocimiento. Tienen que cumplir todas las características que cumplen las US (estimable, realizable en un sprint y su cumplimiento es demostrable)

Estimaciones de Software

Estimar es predecir en un momento donde no tenemos gran conocimiento, donde hay incertidumbre.

Por definición, una estimación no es precisa, tiene un alto nivel de incertidumbre, la cual es mayor mientras más cerca del inicio del proyecto estemos. Es necesario ajustar las estimaciones a medida que avanzamos en el proyecto, ya que tenemos más información.

ESTIMAR NO ES PLANEAR y no necesariamente nuestro plan tiene que ser de igual forma que lo estimado. Si bien sirve para planificar, los planes no tienen que seguirla, ya que no son compromisos, pero se debe tener en cuenta que mientras mayor sea la diferencia entre lo que estimamos y lo que planeamos, más riesgo va a haber.

Para que estimamos?

Estimamos para predecir el futuro, de la forma más certera posible. Se busca predecir cuanto costo, cuanto esfuerzo y cuánto tiempo nos va a llevar hacer un producto.

Una estimación le es útil al equipo de trabajo y no deben ser tomadas como un compromiso. Son tomadas como base para realizar planificaciones y sirven para ver si un producto es factible o no.

Errores en las estimaciones

- Si el proyecto no está organizado, el nivel de conocimiento sobre el producto es menor y eso hace que sean imprecisas las estimaciones
- Si no tenemos en claro cuáles son los recursos con los que la empresa cuenta para desarrollar ni las capacidades de estos.
- Las técnicas también pueden ser fuentes de error, por lo que se recomienda usar más de una y comparar sus resultados.

Que estimamos?

Lo primero que se estima es el **tamaño** del software. Para esto, se utiliza la técnica de contar.

- Contar funcionalidades
- Contar requerimientos
- Contar la cantidad de US
- Contar la cantidad de casos de uso

A su vez, sirve usar datos históricos para comparar el tamaño posible de nuestro software con algún software ya desarrollado.

Otra de las estimaciones que se realiza es el **esfuerzo**. Con esfuerzo nos referimos a la cantidad de horas lineales que se van a necesitar para construir software. No se incluye las personas que lo hacen, ni calendario ni si las actividades son paralelas o no, solo se estima cuantas horas lineales de desarrollo se van a tener.

Luego de estimar el esfuerzo, se estima el **calendario**, que es justamente pasar a días, meses, las horas lineales estimadas, y proponer así una fecha de entrega. Acá se toman las horas lineales y se verifica que se pueden hacer de forma secuencial, se toma cuantos trabajadores tenemos, cuantas horas trabajan por día, etc.

Finalmente, se estiman **costos y presupuestos**.

Técnicas de estimación basadas en la experiencia

Datos históricos

Implica comparar un proyecto nuevo con uno anterior que me permita hacer la estimación de mi proyecto. Los datos básicos concretos que voy a necesitar para tomar como entrada a mis estimaciones son el tamaño, el esfuerzo, el tiempo y los defectos.

El principal problema, es que los dominios analizados pueden ser muy distintos.

Juicio experto

Es la técnica mas utilizada. Se basa en la experiencia y conocimiento de expertos en la materia para determinar la estimación de esfuerzo y tiempo requerido para un proyecto de software. Primero se identifican los expertos y luego estos proporcionan estimaciones basadas en su experiencia y conocimiento.

Se basa en la subjetividad y experiencia de los expertos, por lo que pueden variar mucho.

Puede ser puro, donde un experto estudia las especificaciones y hace su estimación, con la desventaja que si desaparece el experto la empresa deja de estimar.

Otra forma de implementación es WIDEBAND DELPHI, donde un grupo de personas son informadas y tratan de adivinar lo que costara el desarrollo tanto en esfuerzo como en duración. Las estimaciones se recopilan de forma anónima y se envían a expertos para que las revisen. La desventaja es que requiere mas tiempo y recursos que el juicio puro, ya que implica mas rondas de estimaciones y análisis.

Analogía

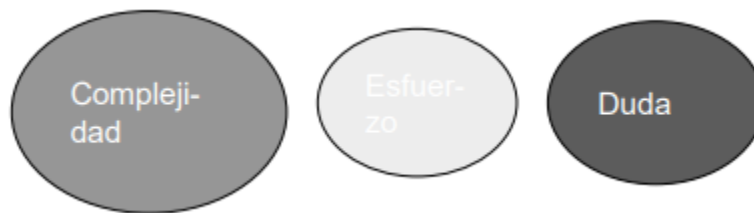
Se basa en los datos históricos, pero se seleccionan específicamente aquellos proyectos que son similares al proyecto actual para realizar la comparación y estimación.

Estimaciones ágiles

Se debe recordar que las estimaciones no deben ser usadas como compromisos, ya que son muy peligrosas y perjudiciales para la organización. Lo mas beneficioso es el proceso de hacerlas y podrían servir como una gran respuesta temprana sobre si el trabajo planteado es factible o no.

En estimaciones ágiles, a diferencia de las tradicionales, se trabaja con **features** o **stories**.

Estas van a ser estimadas usando una medida del tamaño relativo conocida como Story Points. Los Story Points son una ponderación que se le da a la US, cuyo peso es una combinación de su **incertidumbre**, **esfuerzo** y **complejidad**.



Se trata de medidas o estimaciones **relativas**. Ya que **las define el equipo** y no son comparables entre distintos equipos (desventaja).

Uno de los aspectos principales de la estimación relativa es realizar ésta a partir de la **comparación**. Esto es posible a través de una US canónica, no necesariamente la más chica, que será usada como base para definir los Story Points.

El tamaño es una medida de la cantidad de trabajo necesario para producirla o completarla, y a su vez, indica cuan **compleja** y grande es.

Las estimaciones basadas en el tiempo son más propensas a errores debido a factores externos como las habilidades de cada persona, conocimiento del dominio, experiencia, etc.

NO CONFUNDIR TAMAÑO CON ESFUERZO.

El esfuerzo tiene que ver con horas lineales.

Velocity

Es una medida del progreso de un equipo. Se calcula sumando el número de Story Points que el equipo completa durante la iteración. Se cuentan los Story Points de las User Stories **completas**. La velocidad, corrige los errores de estimación.

Es el insumo que sirve para determinar la capacidad (X Story Points), para ver cuantas US va a realizar el equipo en la siguiente iteración. En los primeros sprints, suele ser muy inestable (el equipo está aprendiendo a trabajar juntos).

Poker Estimation

Es una técnica que combina opinión de experto, analogía y desagregación para la asignación de Story Points. Se usa la serie de Fibonacci, ya que crece de forma exponencial, al igual que la complejidad de una US. Los participantes expertos son los desarrolladores.

La técnica de Poker Planning utiliza cartas, donde cada miembro selecciona una que representa el valor que esa persona le asigna a la US y la coloca boca abajo sobre la mesa. Una vez que todos los miembros eligieron su carta, se dan vuelta simultáneamente. Los miembros del equipo deben explicar el por qué de su razonamiento, para luego, al llevar a cabo la siguiente ronda, que todos sepan su pensamiento. Cuando se llega a un consenso, se asigna el puntaje para la US.

Los prerequisites son:

- Lista de features/stories a ser estimadas
- Cada estimador debe tener un mazo de cartas

El Product Owner está presente, da la información necesaria para las estimaciones, pero no participa ya que no va a formar parte del desarrollo.

Si se puntúa una US con complejidad 0, quiere decir que hay desconocimiento sobre esta y es necesario detallarla.

Cuando hablamos de agile, hablamos de empirismo, por lo tanto, en las estimaciones también tenemos que cumplir con las características y pilares del empirismo, que son ejecutar, inspeccionar y adaptar, a través de retroalimentación.

Gestión de productos

En un principio, cuando creamos productos, lo hacemos para:

- Satisfacer a los clientes
- Tener muchos usuarios logueados
- Ganar mucho dinero

Además de estas razones básicas, también tiene que ver con una visión de cambiar el mundo o nuestra diaria.



En la base de la pirámide se ubican las características que si o si tienen que existir para que el software funcione y cumpla su propósito. Estos 3 aspectos, son los que esperamos cuando construimos o cuando vamos a usar un producto de software.

En el tope aparecen otros, relacionados con otras cuestiones de la visión del producto que son difíciles de alcanzar. Se habla de un producto conveniente, placentero y significativo.

El desafío principal tiene que ver con cómo hacemos para construir un producto de software donde **desechamos el desperdicio** y logramos abarcar los aspectos que estamos persiguiendo cuando creamos el producto de software.

Inicialmente, nos vamos a centrar en construir el mínimo producto, para saber si el producto que vamos a construir va a cubrir las expectativas del usuario o las propias del desarrollador del producto.

La idea es plantear una **hipótesis** y con un **mínimo** desarrollo y un **mínimo** esfuerzo, validarla, para reducir ese desperdicio lo máximo posible.

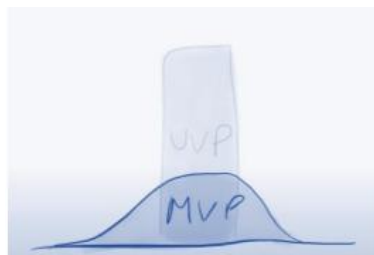
Para esto, se puede utilizar la técnica UVP, que se centra en comprender las necesidades, deseos y problemas de los usuarios y crear soluciones que las solucionen de manera efectiva.

En un principio, nos vamos a imaginar una hipótesis o un valor que tiene el producto o servicio que queremos desarrollar. Mi visión del producto va a ser resolver esa hipótesis. Esto es así ya que si en lugar de solo buscar verificarla, comenzamos a hacer algo completo con muchas funcionalidades y recursos, corremos el riesgo de que el producto no sea algo que el mercado o los usuarios estén demandando.

Es por esto por lo que se busca **minimizar el esfuerzo**.

MVP (Minimum Viable Product – Producto Mínimo Viable)

La idea es trabajar con lo mínimo del producto que necesito obtener. Lo mínimo que este debería tener para que los (posibles) usuarios lo puedan evaluar y me puedan decir si el producto los atrae o no.



El objetivo del MVP es comprobar que la hipótesis del producto a construir funciona o si tengo que hacer cambios. La idea es iniciar un circuito de aprendizaje a partir de la participación de los usuarios y su retroalimentación. A partir de esto, es posible ver si el producto cumple sus expectativas, si sirve o si se tiene que hacer cambios.

La idea no es vender el MVP, si no validar la hipótesis.

Si con cada feedback que recibo de los clientes, recibo algo distinto, quiere decir que quizás no esta tan claro el mercado en el cual este puede funcionar, y significa que tengo que seguir trabajando sobre la visión del producto para encontrarlo.



Otra alternativa que también puede suceder es que en algún punto, la hipótesis tenga variaciones. En este caso, esta puede moverse según la exigencia del mercado. Mi MVP se va a ir moviendo hacia el foco en el cual los clientes se van a interesar. En estos casos, debemos redefinir el MVP en base al conocimiento adquirido, creando un nuevo MVP.



Una vez encontrado el MVP exitoso, es decir, que el feedback y la investigación son exitosas, surge el MMF (Minimum Marketable Feature – Característica Mínima Comercializable)

MMF (Minimum Marketable Feature – Característica Mínima Comercializable)

Su objetivo es definir los elementos mínimos que deben estar presentes en un producto o servicio para que sea comercializable. Es la validación del producto en términos de comercialización.

Ya no se está parado en una hipótesis validando si el producto tiene mercado o no, si no que se está definiendo cual es la pieza mínima que voy a tener que construir y comercializar para que este sea rentable.



Esto sirve para ahorrarnos construir un producto completo, que puede no tener comercialización.

También es importante que se construya en cuestiones mínimas para sacarlo rápidamente a la venta cuando el producto es atractivo y todavía no llegó al mercado, ya que cuanto más tiempo tardo, más riesgo hay de que la idea sea implementada antes. Se sigue con la idea del menor esfuerzo posible, para evitar desperdicios.

El objetivo es asegurarnos de que este primer MMF sirva, con la diferencia de que ahora se habla de una feature (más chica que un MVP). Es lo mínimo que se puede sacar a la venta.

MVF (Minimum Viable Feature – Característica Mínima Viable)

Se trata de una característica mínima que se puede construir o implementar rápidamente, utilizando los recursos mínimos, para probar su utilidad.

El objetivo es validar cual es la característica que hace la diferencia y que va a hacer en la primera medida que compren mi producto.

Es una feature que buscamos que por sí misma ofrezca un valor agregado. Pasamos de centrar la hipótesis del MVP en un producto, a centrarla en una característica de este.

Si esta resulta exitosa, se pueden desarrollar más MMF en esta área para tomar ventaja.

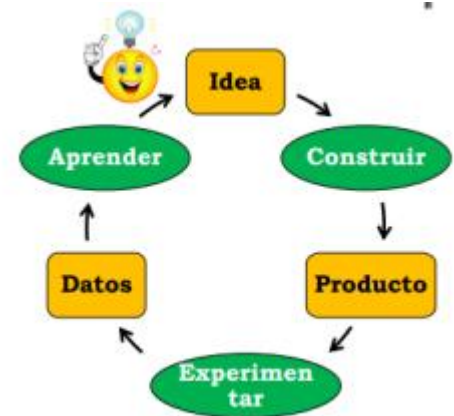
MRF (Minimum Release Feature – Características Mínimas del Release)

Se trata del release del producto que tiene el conjunto de características más pequeñas posible. El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales. El MVP inicial se puede considerar el primer MRF.

Ciclo de feedback o de aprendizaje

Primero se tiene una idea (hipótesis), se construye el producto o la parte que nos permita validarla, luego se **experimenta**, se trabaja con clientes y usuarios y se obtienen datos que son usados como feedback y para ver si es aceptada la hipótesis o no, y a partir de esos datos aprendemos y comenzamos nuevamente el ciclo.

La idea es trabajar con este ciclo para terminar con el producto que el mercado necesita, no el que se me ocurrió inicialmente.



Dilema de “Audacia cero”

Se debe pensar en términos de la construcción de un producto grande, aunque resulte más peligroso. Si se pospone la experimentación con el MVP, van a surgir resultados como una gran cantidad de trabajo desperdiciado, pérdida de información y el riesgo de construir algo muy poco significativo.

Es por esto por lo que se debe usar el MVP para experimentar con los primeros usuarios para verificar la hipótesis.

Supuestos de “Saltos de fe”

Los elementos más riesgosos del plan de un nuevo producto se denominan supuestos de salto de fe. La mayoría de las personas no conocen una determinada solución (o incluso un problema), pero una vez que experimentan con la solución, no podrían imaginarse como vivirían sin ella.

Preparación de un MVP



Características importantes de un MVP

1. Diseño (adecuado y con buena UX)
2. Usabilidad (tiene el suficiente valor para que la gente lo quiera usar)
3. Confiabilidad
4. Funcionalidad (tiene las funcionalidades para solucionar un problema)

MLP (Minimum Loveable Product – Producto Mínimo Adorable)

Es la versión más simple de un producto que, además de ser viable, logra generar una conexión emocional con los usuarios. Su objetivo no es solo validar hipótesis como el **MVP**, sino también **deleitar al usuario desde el inicio**, fomentando la lealtad y la adopción temprana.

Busca enamorar al cliente con el producto. Debe ser algo que el cliente desee tener. Enfatiza la satisfacción del usuario y la diferenciación.

¿Por qué crearlo?

- Reputación de marca: demuestra que la empresa se preocupa por las necesidades reales.
- Productos destacados: permite competir y diferenciarse en mercados consolidados.
- Ciclo de retroalimentación: captar usuarios desde el principio asegura feedback útil y continuo.

Claves para definir un MLP

- Conocer al público objetivo.
- Definir una propuesta clara de valor y diferenciación.
- Resolver al menos una necesidad del usuario de forma **excelente**.
- Añadir un factor sorpresa en lo visual o funcional.

Cómo crearlo

- Preguntar directamente qué valoran los clientes.
- Analizar el mercado y enfocarse en necesidades clave.
- Definir un propósito claro.
- Considerar todos los puntos de contacto con el cliente (no solo las funciones).
- Diseñar con visión de largo plazo.
- Invertir esfuerzo y recursos en calidad desde el inicio.
- Medir el “cariño” que despierta el producto.

Elementos clave de un MLP

- Funcionalidad: resolver el problema del usuario.
- Usabilidad: interfaz intuitiva, sin fricciones.
- Deleite: evocar emociones positivas y fomentar la lealtad.

Comparación:

- MVP: se centra en validar la idea con lo mínimo funcional.
- MMP: es un producto listo para salir al mercado con un conjunto básico de funcionalidades.
- MLP: prioriza la experiencia y la conexión emocional con los usuarios, incluso si tarda más que un MVP en salir.

MVP vs. MLP

Producto Mínimo Viable (MVP). Se centra en lograr su función principal para que el concepto del producto pueda validarse de la manera más económica y rápida posible. Este suele ser el enfoque predilecto de las empresas que tienen prisa por lanzar un producto al mercado.

Producto Mínimo Adorable (MLP). Se centra en características que no solo son funcionales o útiles, sino que también son lo suficientemente impactantes como para deleitar a los usuarios. Esto requiere una investigación de usuarios sobre lo que los clientes consideran valioso y, por lo tanto, su lanzamiento llevará más tiempo que un MVP.

MMP vs. MLP

Producto Mínimo Comercializable (MMP). Es un producto listo para su lanzamiento al mercado. Ya es capaz de cubrir una amplia gama de funcionalidades, atendiendo a grandes bases de usuarios, e incluso podría ser una versión posterior de un MVP anterior (aunque probablemente aún necesite ajustes de UX/UI).

Producto Mínimo Adorable (MLP). Es un producto que prioriza la interacción del usuario y enfatiza la creación de una conexión emocional con los clientes que lo prueban. Esto contrasta marcadamente con los MMP, que se centran en un mayor atractivo para el mercado y la disponibilidad para el lanzamiento.

Software Configuration Management (SCM)

Es una actividad de soporte transversal a todo el proyecto que aplica dirección y monitoreo administrativo y técnico, y cuyo propósito es establecer y mantener la integridad del producto a lo largo de todo el ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

Permite lograr integridad en el producto frente a los cambios y evolución del software.

Es una disciplina que aplica dirección y monitoreo administrativo y técnico a:

- Identificar y documentar las características funcionales y técnicas de los ítems de configuración
- Controlar los cambios de esas características
- Registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos

Se dice que se mantiene la integridad de un producto de software cuando este:

1. Satisface las necesidades del usuario
2. Permite la rastreabilidad durante todo su ciclo de vida (existen vínculos entre los ítems de configuración, que deben ser definidos desde un primer momento).
3. Satisface los criterios de performance y RNF
4. Cumple con expectativas de costo

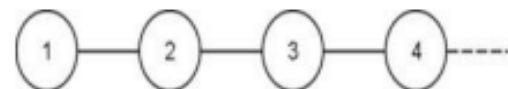
Conceptos clave para la gestión de configuración de software

Ítem de configuración

Son todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos saber su estado y evolución.

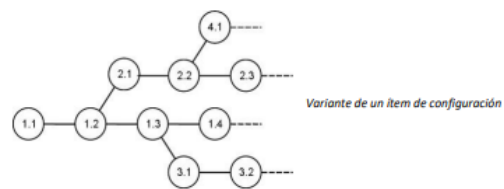
Estos deben poder ser guardados en un repositorio y deben componer toda la información producida como parte del proceso de ingeniería de software. Dependiendo su naturaleza, pueden durar todo el ciclo de vida del proyecto, producto o sprint.

Una **versión** se define, desde el punto de vista de la evolución, como la forma particular de un artefacto (ítem de configuración) en un instante o contexto dado. El control de versiones se refiere a la evolución de un único ítem de configuración o de cada ítem por separado. La versión es un punto en particular en el tiempo de ese ítem de configuración.



Evolución lineal de un ítem de configuración

Una variante es una versión de un ítem de configuración que evoluciona por separado. Pueden representar configuraciones alternativas (por ejemplo, dependiendo del S.O. que se use)



Repositorio

Contiene los ítems de configuración, mantiene su historia, atributos y relaciones. Se usa para hacer evaluaciones de impacto de los cambios y pueden ser una o varias bases de datos.

- Centralizados: un servidor contiene todos los archivos con sus versiones, los administradores tienen mayor control sobre el repositorio y si falla el servidor, falla todo.
- Descentralizados: cada cliente tiene una copia exactamente igual del repositorio completo, si un servidor falla, solo es cuestión de copiar y pegar. La desventaja es que es más complicado llevar un control sobre este.

Presenta dos acciones fundamentales, check-in (llevar al repo) y check-out (sacar del repo).

Líneas base

Es una configuración (conjunto de ítems de configuración) que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo. Sirven como base para desarrollos posteriores y pueden cambiarse solo a través de un procedimiento formal con control de cambios, dirigido por un comité de control de cambios, que permite definir el procedimiento a seguir para manejar estos cambios.

Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.

Se acuerdan parámetros para determinar que se considera o que deben cumplir los ítems de configuración para considerarse como línea base.

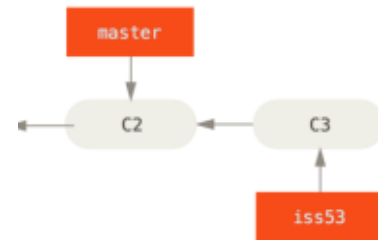
Sirven para tener un punto de referencia, para hacer Rollback o saber que se pone en producción. Nos permite saber cuál era la última situación estable en un momento de tiempo y como se fue evolucionando.

- De especificación: son las primeras líneas base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento.
- Operacionales: contiene una versión de producto cuyo código es ejecutable. La primera línea base operacional corresponde al primer Release.

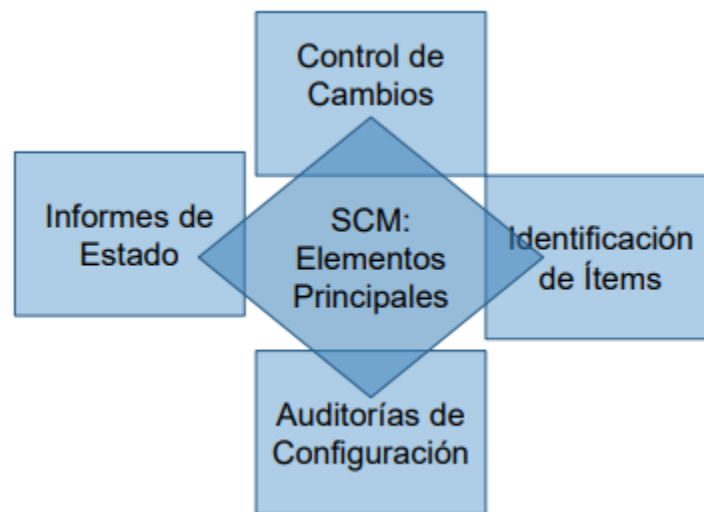
Ramas

Son un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos, ya sea experimentación, resolución de errores o para desarrollar un mismo software para distintas plataformas.

Pueden ser descartadas o integradas (merge) a otras ramas. Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas.



Actividades fundamentales de la SCM



Todas estas actividades son realizadas tanto en el ambiente ágil como en el tradicional. La gestión de configuración debe ser planificada.

Identificación de los ítems

Identificación unívoca de cada ítem, convenciones y reglas de nombrado. También se incluye la definición de la estructura del repositorio y ubicación de los ítems dentro del repositorio.

Además, es importante a la hora de definir los ítems, la duración de su integridad, ya que difieren en función del tiempo que es necesario mantenerlos.

- Ítems de producto: tienen el ciclo de vida más largo y se mantienen mientras el producto exista
- Ítems de proyecto: el plan de proyecto, el listado de defectos encontrados
- Ítems de iteración: conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este

Control de cambios

Tiene origen en un requerimiento de cambio a uno o varios ítems de configuración que se encuentran en una línea base. Es un procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio.

Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios. Existe un comité, formado por representantes de todas las áreas, que es el encargado de reunirse para autorizar la creación y cambios sobre la línea base.

En la primer etapa se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem. El comité luego realiza un análisis de impacto del cambio y decide si se autoriza la propuesta de cambio o no. En caso de que se acepte, se genera una orden de cambio que define que es lo que se va a realizar, las restricciones y los criterios para revisar y auditar. Posterior a la realización del cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y notificar a los interesados sobre los cambios realizados.

Auditoria de gestión de configuración

Son controles autorizados a realizar por el equipo de desarrollo en un momento dado, donde el auditor externo al equipo (independiente y objetivo) analiza las líneas base, para congelar y analizar, en un momento determinado, cual es el estado del software y si se están cumpliendo todas las pautas que plantea la disciplina de SCM.

Es objetiva, ya que hay independencia de criterio (el auditor no tiene condicionamiento respecto a lo auditado). Se hace mientras el producto se esta construyendo y su objetivo es que el mismo tenga calidad e integridad.

- Auditoria física: asegura que lo que está indicado para cada ítem en la línea base o en la actualización se ha alcanzado realmente.
- Auditoria funcional: controla que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos

Sirve a dos procesos básicos, la validación y verificación.

Validación: se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.

Verificación: implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base.

Las auditorias permiten visualizar si se están satisfaciendo los requerimientos y si se ha cumplido la intención de la línea base anterior. Ante un análisis, el auditor puede detectar defectos y ajustar correcciones.

Informes de estado

Mantiene registros de la evolución del sistema y de donde está ubicado el software, comparado con lo que esta publicado en la línea base. Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos. Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.

Su principal objetivo es asegurarse que la información de cambios le llegue a todos los involucrados.

Plan de gestión de configuración

El Plan de Gestión de Configuración define cómo se organizará el control de cambios y versiones en un proyecto: incluye reglas de nombrado de ítems, herramientas a usar, roles (incluido el comité de control de cambios), procedimientos formales, formularios y auditorías.

SCM en Agile

Sirve a los practicantes (equipo de desarrollo) y no viceversa. Hace seguimiento y coordina el desarrollo, en lugar de controlar a los desarrolladores.

Responde a los cambios en lugar de evitarlos y se busca automatizar tanto como sea posible. Se debe eliminar el desperdicio (no agregar nada más de lo que da valor) y debe existir un feedback continuo y visible sobre calidad, estabilidad e integridad.

Los equipos ágiles son auto organizados, por lo que las auditorías de configuración no son una actividad propia de la gestión de configuración en ambientes ágiles.

SCRUM

Es un marco de trabajo ágil que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables a problemas complejos. No es un proceso ni una metodología.

Se basa en el empirismo, que afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa. Busca reducir desperdicios y centrarse solo en lo esencial.

Es binario, no existe un porcentaje de completado en SCRUM, algo está hecho o no lo está. Se usa el Definition Of Done y en la sprint review solo se presentan características que lo cumplieron, las que no, son enviadas al Product Backlog por el Product Owner.

Combina 4 eventos formales para la inspección y adaptación dentro de un evento contenedor, el Sprint. Estos eventos funcionan porque implementan los pilares del empirismo (transparencia, inspección y adaptación).

Valores de Scrum

El éxito en el uso de Scrum depende de que las personas cuenten con 5 valores

- Compromiso (a lograr objetivos y apoyarse mutuamente)
- Enfoque (en el trabajo del Sprint)
- Apertura (el equipo y sus partes interesadas deben estar abiertas sobre el trabajo y los desafíos)
- Respeto (los miembros del equipo se respetan mutuamente para ser capaces e independientes)
- Coraje (los miembros tienen el valor para hacer lo correcto y de trabajar en problemas complejos)

Categorías de responsabilidades

La unidad fundamental de Scrum es un pequeño equipo de personas. Consta de un Scrum Master, un Product Owner y desarrolladores. No existen sub-equipos ni jerarquías.

Los equipos son multifuncionales, es decir, los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint. También son autogestionados, lo que significa que de forma interna, definen quien hace que, como y cuando.

El grupo es lo suficientemente pequeño para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un sprint.

Todo el equipo es responsable de crear un incremento valioso y útil en cada sprint.

Product Owner

Es un rol no técnico, tiene que conocer todo sobre el producto o negocio.

Es quien tiene el poder de tomar decisiones (priorizar) para maximizar el valor resultante del trabajo del equipo. Es único por producto y es quien representa a todos los Stakeholders. Es capaz de cancelar un sprint cuando el objetivo del mismo se vuelve obsoleto.

Es la persona encargada de la gestión de la pila de producto (product backlog), lo que quiere decir que debe:

1. Desarrollar y comunicar explícitamente el objetivo del producto
2. Crear y comunicar los elementos del trabajo pendientes
3. Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido por todos.

Este trabajo es su responsabilidad, lo que no quiere decir que el es siempre el que se encarga de hacerlo, lo puede delegar. Todos deben respetar sus decisiones.

No puede ser la misma persona que un Scrum Master, ya que ese es un rol técnico. Tampoco se puede asumir que porque alguien sepa escribir US, puede ser Product Owner.

Scrum Master

Es quien vela por que las reglas de Scrum se cumplan tal cual como son definidas en la guía de Scrum. Es quien coordina y orquesta las ceremonias de Scrum y es el responsable de la efectividad del equipo. No es un jefe, ya que los equipos son autoorganizados.

Desarrolladores

Son los creadores de producto de trabajo. Tienen la capacidad de tomar una US, comprometerse y terminarla. Los equipos son autoorganizados, es decir, todos tienen libertad en la toma de decisiones, pero es muy importante que estén **comprometidos**.

Eventos de Scrum

Todos los eventos son una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum. Si no se realizan según lo que esta escrito, se pierden oportunidades para inspeccionar y adaptarse. De manera óptima, todos los eventos se llevan a cabo en el mismo tiempo y lugar, para así reducir la complejidad.

El Sprint

Son eventos de longitud fija (Timebox). Un nuevo Sprint comienza inmediatamente al final de la conclusión del Sprint anterior.

Todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la planificación (Sprint Planning), Daily Scrums, Revisión del Sprint y la Retrospectiva, ocurren dentro del Sprint.

Es importante destacar que durante el Sprint, no se aceptan cambios que pongan en peligro el objetivo del Sprint, la calidad no disminuye y el alcance se puede clarificar y renegociar con el Product Owner a medida que se aprende más.

Permite previsibilidad al garantizar la inspección y adaptación del progreso hacia un objetivo del producto, como mínimo, una vez al mes.

Se debe evitar que el horizonte del Sprint sea demasiado largo, ya que el objetivo de este puede volverse obsoleto, complejo y riesgoso. Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un periodo de tiempo más pequeño.

Planificación del Sprint (Sprint Planning)

Es la ceremonia que inicia el Sprint, estableciendo el trabajo que se realizará en el mismo. Este plan resultante es creado por el trabajo colaborativo de TODO el equipo Scrum.

El Product Owner se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendientes de producto más importantes y cómo se relacionan con el objetivo del producto.

Se debe abordar temas como:

- Objetivo del Sprint (Sprint goal)
- Que elementos del Product Backlog se incluyan en el Sprint actual
- Que trabajo se realizará para crear un incremento que cumpla con la Definition of Done. Se descomponen elementos de trabajo en otros más pequeños.

El conjunto de estos 3 temas, se conocen como el trabajo pendiente del Sprint (Sprint Backlog).

Scrum diario (Daily Scrum)

Tiene como objetivo inspeccionar el progreso hacia el objetivo del Sprint y adaptar el Sprint Backlog según sea necesario. Es un evento breve para los desarrolladores del equipo, que se lleva a cabo al mismo tiempo y lugar todos los días laborables del Sprint.

Los desarrolladores pueden seleccionar cualquier estructura y técnicas que deseen, siempre y cuando su Scrum diario se centre en el progreso hacia el objetivo del Sprint y produzca un plan accionable para el siguiente día de trabajo.

Mejoran la comunicación, identifican impedimentos y depende mucho de la transparencia de los desarrolladores.

Revision del Sprint (Sprint Review)

Tiene como objetivo inspeccionar el resultado del Sprint y determinar futuras adaptaciones. El equipo presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso realizado hacia el objetivo del Producto.

Se realiza al final del Sprint y se revisa lo que se logró. Es una sesión de trabajo y el equipo debe evitar limitarla a que se convierta en una simple presentación.

Retrospectiva del Sprint (Sprint Retrospective)

Tiene como objetivo planificar formas de aumentar la calidad y la eficacia. En esta ceremonia, el equipo inspecciona como fue el último Sprint. Se identifican las suposiciones que desviaron el Sprint y se exploran sus orígenes.

El equipo analiza que fue bien, que problemas se encontraron y como esos problemas fueron o no fueron resueltos. Concluye el Sprint.

Artefactos de Scrum

Representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Cada artefacto contiene un **compromiso** para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso.

Product Backlog

Es una lista emergente y ordenada de lo que se necesita para aportarle valor al producto. Los elementos de trabajo pendiente de producto que pueden ser hechos por el equipo dentro del Sprint se consideran listos para su selección en la planificación del Sprint.

Por lo general, recién adquieren esta transparencia luego de ser refinados. El refinamiento es el acto de descomponer y definir aun mas los elementos de trabajo pendiente del producto en artículos más pequeños y precisos.

Dentro de este, se encuentra también el **Product Goal**, que describe un estado futuro del producto, que puede servir como objetivo para que el equipo planifique. Este objetivo es a largo plazo.

Sprint Backlog

El trabajo pendiente del Sprint se compone de un objetivo de Sprint (por qué), el conjunto de elementos de trabajo seleccionados para el Sprint (qué) y el plan accionable para entregar el incremento (cómo).

El **Sprint Goal** es el único objetivo para el Sprint. Este se crea durante el evento de Sprint Planning y luego se agrega al trabajo pendiente del Sprint. Es usado por los desarrolladores a medida que trabajan en el Sprint.

Incremento

Es un paso concreto hacia el objetivo del producto. Cada uno se suma a los anteriores y debe estar completamente verificado, asegurando que todo funcione en conjunto. Para aportar valor, el incremento debe ser utilizable.

Durante un sprint pueden generarse varios incrementos, cuya suma se muestra en la revisión del sprint. No obstante, solo se entregan a las partes interesadas al final.

Un incremento se crea en el momento en que un elemento del Product Backlog cumple con la **Definición de Hecho**, lo que garantiza transparencia y una comprensión compartida del trabajo realmente completado.